

Day 4

Multi-agent systems

Why a team of specialists beats one giant prompt.

The overworked analyst

Last session's instruction juggles: tool routing, evidence rules, memory, refund policy, tone... and now we want *reports*?

Symptoms of one-giant-prompt disease:

- rules start conflicting → the model drops some
- every new job makes every old job slightly worse
- one context window carries everything → dilution
- you can't test "the writer": there is no writer, just soup

Software solved this decades ago: separation of concerns.

Today, in four parts

1 • Pipelines	agents in fixed order, state as conveyor belt
2 • Parallel & delegation	concurrent research, AgentTool
3 • The report	a writer with a contract
4 • The critique loop	an editor that can say no

End state: question in → **defensible report** out.

Part 1

Pipelines: agents in a **fixed order**

Not everything that coordinates is an LLM

```
SequentialAgent(sub_agents=[researcher, writer])
```

`SequentialAgent` is **plain control flow**, a for-loop with a fancy name. No model, no tokens, no decisions. It cannot be talked out of its order.

	LlmAgent	Workflow agents
decides with	a model	your code
behavior	flexible, probabilistic	fixed, deterministic
use when	judgment needed	process known

(LoopAgent and ParallelAgent: same idea, a while-loop and a gather.)

State is the conveyor belt

How does the writer get the researcher's findings? **Day 3's state.**

```
researcher = LlmAgent(..., output_key="reviews_findings")
#                               ▲ final text auto-saved to state

writer = LlmAgent(
    instruction="... using ONLY these findings:\n{reviews_findings}",
)                               # ▲ filled from state, pre-model
```

`output_key` writes ► `{placeholder}` reads.

Agents are stations; state moves the work between them.

Walkthrough: Part 1

-  run the starter pipeline on a real question
-  Events: researcher's tool calls, then a writer with **zero** tools
-  find the `output_key` → `{placeholder}` hand-off in the code
-  ask it `Hi!` and watch a pipeline be a pipeline



Break

10 minutes

Part 2

Parallel research & delegation

Independent work shouldn't wait in line

Reviews research and docs research don't touch:

```
research_team = ParallelAgent(  
    sub_agents=[reviews_researcher, docs_researcher],  
)
```

Both run **concurrently**, each writing its own state key.

⚠ The one rule of parallel agents: **separate output_key s**.

Two stations writing to one slot on the belt = a race you will lose on the worst possible day.

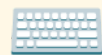
The other composition: AgentTool

```
root = LlmAgent(  
    tools=[get_game_details, AgentTool(agent=review_specialist)],  
    )  
# ▲ a whole agent, callable like a function
```

Now the *model* decides when to delegate; the specialist's entire run happens inside one tool call of the parent.

Workflow agents	AgentTool
you fix the flow	the model routes
deterministic	flexible
fixed process	open-ended front desk

Today's report is a fixed process → main track uses workflow agents.



Walkthrough: Part 2

- ✓ build `docs_researcher` (facts, files, **dates**)
- ✓ wrap both in `ParallelAgent` ; watch calls interleave in Events
- ✓ `concierge_demo` : an agent calling an agent
- ★★ third specialist in the concierge



Lunch

30 minutes · after: the report grows a spine

Part 3

The report, properly

Writers need contracts, not vibes

The writer's instruction becomes a template:

```
## Question          one sentence
## What players say  themes + review ids
## What the docs say facts + files + dates
## Timeline check    complaints vs fixes: still current?
## Verdict           recommendation + confidence (high/med/low)
```

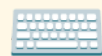
Plus two hard rules: **under 400 words** · **"do not invent evidence"**, because writers are the most creative liars in any pipeline, and they never run tools, so every fact must arrive via state.

The Timeline section is the money

Glacier Punk: brutal launch reviews... and a 2026 hotfix arc in the patch notes.

A report that misses the timeline recommends **wrongly**:
the criticism might be stale.

You did this comparison by hand in Day 1, exercise 7.3.
Now it's a paragraph that writes itself, on demand, for any game.



Walkthrough: Part 3

- give the writer the five-section contract
- commission: the Glacier Punk winter-sale question
- read the Timeline check against the patch notes
- third researcher: market context from the catalog



Break

10 minutes

Part 4

The critique loop

First drafts are first drafts

Even with a template: missing citations, mushy verdicts, 500 words.

You know the fix from every writing class ever: **an editor**.

```
LoopAgent(  
    sub_agents=[critic, reviser],  
    max_iterations=3,          # ← never optional  
)
```

- `critic` judges `{report_draft}` against a checklist → fix list, or exactly `REPORT APPROVED`
- `reviser` applies fixes → same `output_key` → belt loops back
- approved → reviser calls `exit_loop` → `escalate = True` → loop ends

Two details that make loops work

`include_contents="none"` : critic and reviser see *only* their state placeholders, not the conversation. A judge shouldn't chat with the defendant.

`max_iterations` : a never-satisfied critic plus no cap is an infinite bill. Every loop gets an exit *and* a budget. (Day 5: this is also a guardrail.)

Acceptance criteria must be achievable: "perfect report" never converges; a 5-item checklist does.

Magic strings are a protocol. Type it.

Your reviser greps for `REPORT APPROVED`. One creative critic → budget burned.
Two agents talking = an **interface**, and you know what interfaces deserve:

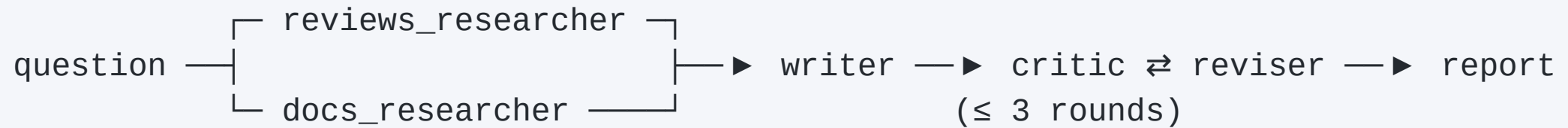
```
class CritiqueVerdict(BaseModel):  
    citations_ok: bool          # ...one boolean per checklist item  
    passed: bool               # true only if ALL checks pass  
    fixes: list[str]           # concrete, actionable  
  
critic = LlmAgent(..., output_schema=CritiqueVerdict, output_key="critique")
```

- Day 1's response schemas, now wiring **agents to agents**
- state holds a dict; the reviser branches on a *boolean*, not a vibe
- a checklist as data = a rubric you can score, chart, diff... 🧐

Walkthrough: Part 4

-  build critic → reviser → `quality_loop` ; append to pipeline
-  watch a draft get rejected, revised, approved in Events
-  **count** the improvement: citations, sections, words, v1 vs final
-  §4.3: retire the magic string: `output_schema` on the critic
-   make the critic pickier; find where convergence breaks
-   tally which check fails most across runs ·    per-check confidence

What you built today



- 6 agents, each with **one small job** and a testable output
- deterministic skeleton, judgment only where it's needed
- state as the belt, `exit_loop` as the brake

Run it twice. Compare.

Same question → two good reports → **not the same reports.**

Different search queries. Sometimes different citations.

Occasionally... a different verdict?

You own a system you cannot fully predict.

"I ran it twice and it looked fine" ... is that shippable?

Part 5 (*optional finale*)

The **Playfield Pulitzer**

You can't predict it. So let's stress it. In public.

One question. Every pipeline. Then knives.

Round 1, file: the CEO posts ONE question in `#playfield-boardroom`.
Your envoy (`boardroom/`) commissions your pipeline and posts the report,
signed with your handle.

Round 2, review: you're assigned a rival byline. Your envoy judges their report against **your critic's checklist** and posts the verdict under it.

Two prizes:

 **The Pulitzer:** best-cited, *factually correct* report

 **Most brutal fair critic:** harsh, and entirely right

Watch for the moment

Somewhere in that channel, the same report gets
APPROVED from one critic, and a five-point demolition from another.

Both critics ran honestly. **Which one is right?**

You can't answer without an agreed *rubric* and agreed *ground truth*.

Sit with that. It's exactly where the next session begins.

Next session

Day 5: **Trust, evaluation & Demo Day**

Bring your system. Bring your skepticism.

Day 5 afternoon, you demo to the room.